

[활동 1] 코흐 곡선(koch) 재귀 함수 구조와 부품 치환 원리

선분 1개를 3등분하여 산 모양 4개 구간으로 바꾸는 재귀 함수의 핵심 빈칸을 채워봅시다.

```
def koch(length, depth):
    # 1. [종료 조건] depth가 0이면 더 이상 쪼개지 않고 진짜 직선 1개 긋기
    if depth == ____:
        forward(length)
    else:
        # 2. 선분 길이를 3분의 1로 축소
        length = length / ____
        # 3. 4개 구간마다 단계(depth - 1)를 줄인 koch 함수 호출
        koch(length, depth - 1)      # 1구간 (평지)
        left(60)                     # 오르막 좌회전
        koch(length, ____ )          # 2구간 (오르막)
        right(120)                   # 내리막 우회전
        koch(length, depth - 1)      # 3구간 (내리막)
        left(60)                     # 수평 좌회전
        koch(length, depth - 1)      # 4구간 (평지)
```

질문 1. if depth == 0 종료 조건이 필요한 이유는?

작성란:

질문 2. koch 함수 안에서 left(60), right(120)을 쓰는 이유는?

작성란:

[활동 2] 줄 번호로 추적하는 1단계 재귀 실행 흐름

정삼각형 첫 번째 변에서 koch(300, 1)을 실행했을 때 컴퓨터의 실제 줄 번호 이동 순서를 살펴봅시다.

koch(300, 1) 실행 시 컴퓨터의 실제 줄 번호 이동 순서:

- ① 5번 줄 (1 == 0 거짓) → 7번 줄 else로 이동 → 8번 줄: length = 100 축소
- ② **9번 줄:** koch(100, 0) 호출 → 5~6번 줄 실행: forward(100) 선 긋기 1
- ③ 10번 줄: left(60) 회전
- ④ **11번 줄:** koch(100, 0) 호출 → 5~6번 줄 실행: forward(100) 선 긋기 2 (오르막)
- ⑤ 12번 줄: right(120) 회전
- ⑥ **13번 줄:** koch(100, 0) 호출 → 5~6번 줄 실행: forward(100) 선 긋기 3 (내리막)
- ⑦ 14번 줄: left(60) 회전
- ⑧ **15번 줄:** koch(100, 0) 호출 → 5~6번 줄 실행: forward(100) 선 긋기 4 (평지)

[추적 질문] 거북이가 화면에 진짜 잉크를 묻혀 선을 긋는 동작은 몇 번 줄에서만 일어날까요?

작성란:

[활동 3] 눈송이 단계별 선분 개수 규칙성 표 채우기

정삼각형(선분 3개)의 각 변이 단계마다 4개로 쪼개질 때, 총 선분 개수 수식 $3 * (4 ** depth)$ 을 계산해 봅시다.

단계 (depth)	한 변의 선분 개수	눈송이 전체 선분 개수	선분 1개의 길이
0단계	1개	3개 (3×1)	300
1단계	4개	12개 (3×4)	100 ($300 \div 3$)
2단계	16개	_____개	약 33.33
3단계	64개	192개 (3×64)	_____

[활동 4] 화면을 지우고 새로 그리는 draw 함수 분석

키를 누를 때마다 이전 그림을 지우고 새로운 단계의 눈송이를 즉시 렌더링하는 draw 함수를 완성해 봅시다.

```
def draw(depth):
    tracer(0, 0)                                # 화면 깜빡임 없이 즉각 렌더링
    _____()                                # 이전 단계의 그림을 깨끗이 지우기

    # 상단 안내 문구 및 현재 단계 선분 개수 출력
    penup(); goto(0, 195)
    count = 3 * (4 ** depth)
    write(f"현재 단계: {depth}단계 | 선분 개수: {count}개", align="center", font=("맑은 고딕", 13,
"bold"))

    # 중앙 위치로 이동 후 정삼각형 3개 변에 koch 함수 적용
    goto(-150, 60); setheading(0); pendown()
    for _ in range(3):
        koch(300, _____)
        right(_____)
    update()                                     # 완성된 화면을 한 번에 출력
```

[확인 질문] draw 함수 안에서 clear() 명령어가 없다면 키보드를 누를 때 화면에 어떤 문제가 생길까요?

작성란:

[활동 5] 키보드 이벤트 연결 (onkey & listen)과 조작 실습

사용자가 키보드 숫자(0~4)를 눌렀을 때 해당 단계를 그리도록 이벤트를 연결하고 실행해 봅시다.

```
# 키보드 번호(0~4)와 draw 함수 연결하기
onkey(lambda: draw(0), "0")
onkey(lambda: draw(1), "1")
onkey(lambda: draw(2), "2")
onkey(lambda: draw(3), "3")
onkey(lambda: draw(4), "_____")

_____()                                # 키보드 입력을 귀 기울여 듣는 스위치 켜기
draw(0)                                    # 처음 실행 시 0단계 기본 화면 출력
done()
```

[실습 관찰] 프로그램을 실행한 후 키보드의 0 → 1 → 2 → 3 → 4 키를 차례대로 눌렀을 때, 눈송이의 테두리는 어떻게 변하나요?

작성란:

[활동 6] 나만의 인터랙티브 눈송이 도안 디자인하기

배경색과 선 색상, 굵기를 변경하여 나만의 멋진 인터랙티브 코호 눈송이를 기획해 봅시다.

디자인 설정 항목	내가 적용할 코드 / 값	디자인 효과 및 컬러 컨셉
배경색 설정	bgcolor("_____") (예: "#1a1a2e", "black")	어두운 밤하늘 / 깔끔한 흰색
선 색상 & 굵기	color("_____"), width(_____)	네온 민트, 골드 / 1.5 ~ 2.5 굵기
가장 마음에 드는 단계	[] 1단계 [] 2단계 [] 3단계 [] 4단계	채색하기 적당한 단계 선택

[도안 한 줄 소개] 내가 만든 코호 눈송이 디자인의 이름과 컬러 컨셉을 작성해 보세요.

도안 이름 및 컨셉: